

Neural Privilege Boundary: Deep Research on the Security Principles and Best Implementation Path

Executive summary

The supplied Neural Privilege Boundary (NPB) plan is unusually strong in one important respect: it does not treat prompt-injection resistance as the same problem as authorisation. It separates three claims that are often incorrectly blended together: deterministic capability enforcement, denied-source noninterference, and policy-state integrity. It also explicitly declines to claim that protected policy memory or prompt formatting can make arbitrary natural-language generation injection-proof, and requires every privileged effect to pass through a trusted host-side broker. That separation should remain the architectural centre of the project. [source document]

The most important conclusion of this research is:

NPB should be engineered first as a small reference-monitor/capability system around an untrusted probabilistic model, and only secondarily as a neural-state security mechanism.

That recommendation follows directly from classical security engineering. A reference monitor is expected to mediate every protected access, be tamper-resistant and be sufficiently small to analyse; least privilege gives each component only the authority necessary for its task. These principles align almost exactly with the plan's immutable host-created profile, narrow disclosure path, typed action proposal and mandatory broker. Early capability systems similarly separated possession of authority from arbitrary computation, while later object-capability work and Macaroons developed practical forms of confined and attenuated authority.

The plan is also well aligned with the actual prompt-injection problem. NIST defines prompt injection as exploiting the concatenation of untrusted input with a higher-trust party's prompt, while indirect-prompt-injection research shows that external content can change model behaviour and, in agentic systems, redirect API/tool operations. The architectural answer is therefore not merely to make the model "understand" which instructions are trusted, but to ensure that even a completely compromised model response lacks ambient authority.

Recent evidence strengthens that conclusion. Instruction-hierarchy training can materially improve resistance to lower-priority malicious instructions, so it is useful defence-in-depth, but it remains a learned behavioural property rather than a deterministic access-control mechanism. A 2025 study found that simple agent/tool "firewalls" could saturate several public prompt-injection benchmarks, yet the authors also demonstrated practical bypasses and argued that benchmark weaknesses could make apparently perfect results misleading. The 2026 AgentDyn work likewise argues that contemporary agent-security benchmarks

require more dynamic and realistic tasks and attacks. This directly supports the plan's insistence that benchmark success not be presented as a proof.

The production recommendation is therefore a two-track architecture:

NPB-Core, which is the security boundary, should physically exclude unreadable evidence before any external provider call, accept only a discriminated structured response, validate it independently, render disclosure from host-owned source material, and mediate every state-changing operation through a trusted broker holding short-lived, principal-bound and argument-bound authority. This is the component on which production security claims should rest.

NPB-Neural, which is a research component, should experimentally investigate attention masking, immutable policy K/V state and denied-source noninterference for specific local model/runtime combinations. It should not sit in the production trusted computing base unless there is a genuine use case that cannot be satisfied simply by omitting denied data.

A useful way to state the final security promise is:

The model may recommend; only trusted code may disclose or authorise.

That formulation is stronger than “the model resists prompt injection”, because its correctness does not require semantic obedience from the model.

The proposed high-level architecture is:

The principal changes I recommend to the supplied plan are:

Make the model return references to disclosure data rather than the bytes ultimately released whenever practical. The trusted host should copy the canonical quote from the versioned source itself. This removes Unicode, encoding and model-supplied-text ambiguity from the disclosure boundary. Use a simple server-side opaque one-time grant for the first broker implementation. Introduce Macaroons or sender-constrained cryptographic tokens only when distributed delegation actually requires them. Macaroons are attractive for attenuation, while DPoP contains useful replay-defence ideas; however, DPoP explicitly does not bind the full request body, so it is not by itself an argument-authorisation mechanism. Treat JSON Schema as syntax/structure enforcement, not semantic authorisation. JSON Schema itself notes that structural validation can be insufficient and that some format

1.

2.

3.

semantics are annotations unless explicitly asserted. Strengthen the neural proof obligation around position independence. Masking denied keys/values is insufficient for exact noninterference if changing the denied source can shift public token positions or alter cache organisation. Either require shape-preserving replacements for the theorem or design independent positional namespaces/segments.

4.

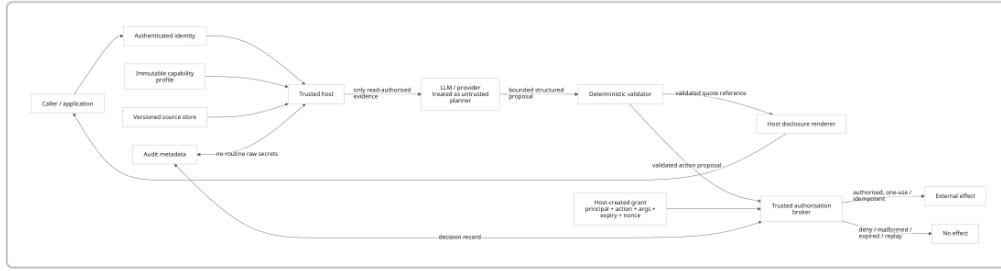


Figure 1: Host-owned capability architecture (reproduced from the supplied report).

Model-check broker concurrency before production. Replay prevention, single-use grants and ambiguous effect completion are state-machine problems, for which TLA+/TLC is a better first tool than a heavyweight proof assistant. Upgrade the locked evaluation beyond static benchmark saturation. InjecAgent and AgentDojo remain useful, but add adaptive attacks and preferably NIST’s AgentDojo-Inspect or a recent dynamic suite. BIPIA is now archived as of 16 September 2026 and, in this project, the sampled cases are already consumed development material.

5.

6.

Overall, I assess the plan as architecturally sound and researchable, provided the deterministic host/ broker path is made the root of trust and the neural mechanisms are described as scoped information- flow experiments rather than a general solution to prompt injection.

Scope, assumptions and threat model

What was available for review

The uploaded plan is the only project artefact supplied in this conversation. It refers to `neural_state_firewall/CAPABILITY_BOUNDARY.md` , `neural_state_firewall/READ_PERMISSIONS.md` and `../docs/neural_state_firewall_paper/STUDY_PROTOCOL.md` , but those referenced files and the source repository were not provided. Consequently, statements about existing implementation behaviour—for example sealed K/V cache behaviour, the exact validator control flow and the historical 1,750-attempt run—are treated here as claims made by the supplied plan, not independently audited implementation facts. [source document]

The historical BIPIA-derived 175-case set is correctly identified in the plan as consumed development material, with no semantic attack-success labels, and the plan explicitly warns against treating 42 changed ordinary outputs as successful attacks. That methodological distinction is correct and should be preserved. [source document] BIPIA itself was designed to study indirect prompt injection across external-content tasks; its repository also warns that WebQA and summarisation source data require their own source-data licensing treatment, and Microsoft archived the repository on 16 September 2026.

Explicit assumptions introduced by this assessment

Assumption Why it matters

The application host, capability validator and broker are trusted and outside the attacker's control.

If the reference monitor itself is compromised, none of the proposed application-layer guarantees survives.

This is what makes prompt injection a model- behaviour problem rather than an access-control bypass.

The model and retrieved content are not trusted for authorisation decisions.

NPB-Core can prevent unauthorised sources from being sent; it cannot cryptographically hide readable plaintext from a provider that must infer over it.

An external provider is trusted for the confidentiality of evidence deliberately sent to it under the provider contract.

A capability profile is meaningless unless the principal-to-profile binding itself is trustworthy.

Caller identity is authenticated upstream using a mechanism outside NPB.

Assumption Why it matters

Otherwise a validated source identifier can become a time-of-check/time-of-use disclosure vulnerability.

Source objects used for disclosure are immutable/ versioned during a request.

The first production scope permits constrained responses such as quotes and typed actions; unrestricted prose remains outside the deterministic disclosure guarantee.

This is the principal utility/security trade-off in the plan.

This permits a simpler opaque, one-use grant design before distributed credentials are needed.

The first broker runs centrally and can maintain durable state.

Therefore hardware cost and workload size can only be estimated qualitatively at this stage.

The exact model families and first privileged action are still undecided.

The roadmap assumes roughly two implementation/security engineers plus a research owner, with access to one suitable evaluation GPU and no major procurement delay.

Different staffing substantially changes elapsed time.

Threat model

The relevant adversarial channels should be stated more explicitly than in a conventional LLM threat model.

The attacker may control some user input, documents, emails, web pages, search results, tool results or other retrieved text; may deliberately cause malformed or adversarial model outputs; may replay previously authorised proposals; may attempt cross-user or cross-instance source references; may race concurrent requests; may seek to exfiltrate non-disclosable material; and may attempt to cause a permitted tool to execute with unautho-

rised arguments. These are credible threats because indirect prompt injection is specifically about low-trust content influencing a model operating in a higher-privilege application context.

The concern is no longer merely academic. NIST's 2025 work on agent hijacking describes malicious instructions in ingested content causing agents to take unintended harmful actions, and NIST's 2026 vulnerability records include systems where prompt injection interacted with tool naming or sandbox boundaries to produce materially dangerous execution paths. These examples do not prove NPB's architecture, but they illustrate why tool privilege must not be ambient model privilege.

The following should be explicitly out of scope for the initial NPB theorem: compromise of the host OS/ kernel or broker signing/storage keys; arbitrary reads of local process memory; malicious GPU firmware; electromagnetic/physical side channels; denial-of-service timing/resource channels; provider compromise after authorised plaintext has been transmitted; and semantic obedience of unrestricted natural-language generation. This matches the plan's intended distinction between information-flow behaviour inside a specified model/runtime and broader system security. [source document]

One important distinction is worth adding:

"Denied content cannot affect model logits" is not the same claim as "denied content is inaccessible to the runtime."

A local process may still tokenise, store or compute K/V values for denied content before an attention mask excludes causal influence. Such a design may establish a noninterference property over chosen observables, while offering no protection against a compromised local runtime reading the original bytes. The theorem and paper should say this explicitly.

Principles and theoretical foundations

Reference monitoring, complete mediation and least authority

NPB-Core is best understood as an application-level reference validation mechanism. NIST's reference-monitor formulation requires mediation of accesses, resistance to tampering and a mechanism small enough to analyse and test. Least privilege requires giving each entity only the system resources and authorisations necessary for its function.

This maps cleanly onto the plan:

$C = (R, D, A)$

$R \cap D \subseteq R \cap A$

where R is the set of sources the instance may read, D is the set from which it may disclose content, and A is a map from permitted action names to their argument constraints. The model does

not receive the ability to modify D . The caller does not get to supply an authoritative A . Every output is

$C \cap C \cap C$

interpreted under the host-owned D .

Saltzer and Schroeder’s foundational security-engineering work is particularly relevant here: least privilege, fail-safe defaults, complete mediation and separation of privilege are architectural principles

rather than properties of any individual classifier. The plan’s “invalid output → release no text and

perform no effect” is a direct application of fail-safe defaults; “every privileged effect → one named broker” implements complete mediation.

Capability-based security adds an even more useful framing. Dennis and Van Horn’s early work developed protected references to computing objects, and later object-capability work focused on composing mutually untrusting components while limiting authority. In NPB terms, the LLM should be treated as such an untrusted component: it can possess descriptions of potential operations without automatically possessing the authority to perform them.

Macaroons are relevant if the architecture eventually requires delegated/attenuated authority: they use chained message authentication codes and caveats to constrain who may exercise authority, where and under what conditions. For the first implementation, however, introducing delegated bearer credentials would enlarge the attack surface unnecessarily. A central opaque grant record is easier to revoke, inspect, consume atomically and reason about.

Structured outputs are a parser boundary, not an authorisation boundary

Using a strict structured-output provider is useful because it changes an unbounded language problem into a bounded parser/validator problem. JSON Schema Draft 2020-12 provides constructs such as

type, enum and const, and structural validation can reject object shapes that violate the schema.

The specification also explicitly notes that structural validation may be insufficient for application semantics and that format is not automatically a semantic authorisation check.

Therefore the recommended action validation pipeline is:

bytes → strict parse → schema → profile → semantic constraints → grant → effect.

A model-native “JSON mode” should therefore be considered a convenience mechanism, not a trusted security control. The host must independently reject duplicate object keys, unexpected properties,

excessive nesting/size, invalid enumeration values, type coercions, unknown action variants and any semantic argument that exceeds the capability.

For actions, use a discriminated union rather than a generic {name, args} bag where possible. A

conceptual schema is:

Proposal := QuoteProposal(source_id, segment_id, optional_span) | SendEmailProposal(recipient_contact_id, subject_id, body_template_id) | CalendarProposal(calendar_id, start, end, attendee_ids) | NoOp(reason_code)

Each variant should have additionalProperties: false or equivalent closed-world validation.

More importantly, each field must subsequently be checked against the host profile and

current principal.

Host-rendered disclosure is stronger than validated model text

The plan proposes allowing only exact quotes from sources marked disclosable. That is already much safer than arbitrary prose. [source document] It can be strengthened further by changing the primitive from “the model supplies a quote” to “the model nominates a host-owned source region”.

For example:

model output: { "kind": "quote", "source_id": "doc-7", "segment_id": "seg-42" }

trusted host: 1. verify $\text{doc-7} \in \text{disclose}(C)$ 2. verify seg-42 belongs to the immutable version of doc-7 3. copy bytes for seg-42 from the canonical source snapshot 4. release those host-owned bytes

This removes several unnecessary questions from the security proof: whether Unicode was normalised consistently, whether visually confusable characters were substituted, whether two identical strings occur at different provenance locations, and whether a model can exploit an inconsistent exact-

substring implementation. Where fine-grained spans are required, the host can expose stable segment IDs and bounded offsets rather than forcing a model to calculate global byte offsets.

A source version hash should be bound to the request or profile snapshot. Otherwise a source could be authorised at validation time and replaced before rendering.

Brokered authority, replay prevention and exactly-once illusions

Every privileged action should require a host-created grant independent of the model’s proposal. A minimal grant record should bind:

grant_id principal_id profile_hash instance_id action_name canonical_argument_hash resource_scope issued_at expires_at consent_record idempotency_key state

The broker accepts an action only when the principal, NPB instance/profile, action and canonical arguments all match the grant.

The plan’s emphasis on expiry and replay protection is well founded. OAuth DPoP demonstrates practical patterns for sender-constraining credentials, including unique proof identifiers, short acceptance windows and replay detection. The standard also makes an important limitation explicit: DPoP does not bind the entire request payload. Consequently, adopting DPoP-like proof-of- possession is not enough for NPB; the specific canonical action arguments must also be bound into the authorisation decision.

Bearer-token practice likewise emphasises short validity periods and protection against token capture/ replay. For a central NPB broker, an opaque server-side grant is simpler still: possession of a model- generated value cannot create authority because only the host can create the corresponding active broker state.

One subtle issue missing from the plan is the impossibility of pretending all remote side effects have clean “exactly once” semantics. Suppose the broker calls an external API, the API commits the operation, but the network connection fails before the broker receives the acknowledgement. Blind retrying may duplicate the effect; blindly declaring failure may

misreport reality. NPB should therefore promise at-most-once authorisation, and use a stable idempotency key whenever the downstream service supports idempotent retries. Ambiguous completion should enter an UNKNOWN state and require reconciliation rather than silently retrying with new authority.

Information-flow noninterference and the neural claim

The theoretical foundation for the plan's denied-source claim is noninterference: changes to protected/ high information should not alter chosen low/public observations. Goguen and Meseguer's 1982 work

is foundational to this approach to information-flow security.

For NPB, let

= policy/prefix state,

- P

= allowed/public evidence,

- A

= denied evidence,

- D

= generated prefix before decoding step ,

- $Y < t$

= public next-token logits.

- L_t

The desired theorem can be written:

,

,

$$L(P, A, D, Y) = t < t L(P, A, D, Y) \forall D, D, t, t$$

< t

under explicitly specified runtime assumptions.

In standard scaled dot-product attention, a token interacts with other sequence positions through attention over query, key and value vectors. If every path from a public/generated query position to all denied key/value positions is truly removed at every layer, and all other public state is independent of , a layer-by-layer induction can establish independence of public hidden states from denied tokens

D

at the mathematical abstraction.

For deterministic greedy decoding, equal logits imply the same first generated token; the identical generated prefix then becomes the induction hypothesis for the next decoding step. For stochastic generation, the clean theorem is equality of output distributions; identical sampled strings additionally require coupling the random-number state.

There is, however, an important condition the current wording needs to strengthen:

Masking denied attention edges does not by itself prove noninterference if changing denied content can change the positions or shape of the public computation.

D D'

Consider replacing by a differently tokenised . If denied material occurs before public material and

changes token count, then public position indices can shift. Transformers explicitly encode token position, so public hidden states may change despite zero attention weight to the denied values. The clean theorem therefore needs one of two assumptions:

$$|\text{Tok}(D)| = |\text{Tok}(D')|$$

for all substitutions covered by the proof, or preferably a segmented representation in which public/ generated positions are assigned independently of denied-source length.

This is one of the most consequential changes I recommend to the research protocol.

Implementation assumptions are equally important. Hugging Face's cache interfaces support multiple cache organisations, including dynamic growth, static caches and model-dependent sliding/chunked

behaviour. A proof for one cache/attention path therefore does not automatically transfer to sliding-window attention, sparse attention, another model adapter or another fused kernel.

Likewise, "exactly unchanged logits" needs two levels of interpretation. At the mathematical model it can mean exact functional equality. GPU execution, however, may contain nondeterministic operations; PyTorch's deterministic mode can require deterministic implementations where supported, but PyTorch explicitly notes that this switch alone is not sufficient to guarantee complete reproducibility. The implementation evidence should consequently include a simple deterministic CPU/reference backend for exact differential tests and separately report GPU/backend conformance.

Immutable policy state does not imply policy obedience

The policy-K/V idea is structurally reasonable. Transformer inference commonly caches key/value tensors for prior tokens so they do not have to be recomputed on every decoding step. A policy prefix can therefore be computed once and its cache made immutable while working-context state is appended elsewhere.

What follows from such an invariant is:

$$\text{PolicyKV}_{t+1} = \text{PolicyKV}_t$$

What does not follow is:

$$\text{GeneratedOutput}_t \models \text{Policy}_t$$

A model can preserve every byte of the system-prefix K/V cache and nevertheless assign greater behavioural weight to an adversarial instruction it is legitimately permitted to attend to. That is an inference directly consistent with the attention architecture and with empirical prompt-injection research showing that readable untrusted content can alter model behaviour.

Instruction-hierarchy training attacks this behavioural problem directly by training models to privilege higher-authority instructions, and published results show significant robust-

ness improvements over baseline behaviour. It should be considered a useful additional layer, but not substituted for host authorisation.

The plan's proposed split between structural integrity and behavioural obedience is therefore exactly the right scientific framing. [source document]

Candidate architectures and comparative assessment

The table below is an engineering synthesis rather than a claim that the methods have been measured head-to-head on one common dataset. Accuracy here means accuracy of enforcing intended application policy while preserving legitimate task completion, not generic LLM factual accuracy. Security ratings are scoped to prompt-injection/capability threats; they do not imply host-compromise resistance.

Candidate method	Accuracy	Maturity	Complexity	Cost	Security	Scalability	Assessment
Prompt delimiters, reminders, system-message formatting	Medium on benign tasks; unreliable adversarially	High	Low	Low	Low as an authority boundary	Very high	Useful ergonomics only. BI evaluation similar prompt approach indirect remain fundamental problem
Instruction-hierarchy / adversarial model training	Medium-high	Medium	High	High during training	Medium; probabilistic behavioural protection	High once trained	Valuable defence depth, but for real evidence not a determined privilege mechanism
Prompt/ tool input-output detector or "firewall"	Medium-high	Medium	Medium	Low-medium	Medium; depends on detector/ attack distribution	High	Can make reduce but add bypass benchmark saturate remain concern
Host capability gate + trusted broker	Very high within narrow typed contract	High underlying security primitives; NPB integration still prototype	Medium	Low-medium	Very high for defined disclosures/ effects if completely mediated	High	Recommend product of trust Match reference monitor least-principles
Neural attention masking + sealed policy K/V	Potentially very high for narrowly proved noninterference	Research	Very high	Medium-high engineering cost	High only for the proved information-flow property; low evidence for semantic obedience	Medium; backend/ model dependent	Keep as separate research Transferring cache implementation make assumptions model/ specific

Candidate method	Accuracy	Maturity	Complexity	Cost	Security	Scalability	Assessment
Process/sandbox isolation + brokered capability boundary	High	High in conventional systems security	High	Medium-high	Very high when all effects cross the trusted boundary	Medium-high	Appropriate when the model can run files/tools; strong evidence contains higher operational Referential semantic remain essential

Why the deterministic capability architecture wins

The decisive security advantage of the host-gate architecture is that **its primary safety property need not depend on correctly classifying the adversarial text.**

A prompt-only defence effectively asks:

“Will the model recognise and ignore this attack?”

A detector asks:

“Will another model/classifier recognise this attack?”

A capability boundary asks:

“Even assuming the model follows the attack perfectly, can the resulting proposal cross this deterministic authorisation boundary?”

For privileged effects, the third is the stronger design.

This does not make prompt-injection research unnecessary. Injection can still destroy utility, make the model refuse work, bias which allowed action it proposes, or persuade a human to grant something unsafe. But it prevents the model's own linguistic interpretation from becoming the final authorisation decision.

Recommended broker credential choices

Broker mechanism Best use Strengths Weaknesses Recommendation
Simple revocation, atomic state, easy audit and replay detection

Opaque server-side one-use grant

First NPB production implementation
Requires central/ shared state Default choice
Revocation/replay state remains difficult; signing/ canonicalisation complexity
Verification without central lookup
Add only when distribution requires it
Signed short- lived grant
Multi-service deployments
Sender- constrained token / DPOP-style proof
Binds use to a key and request endpoint/ method
DPoP does not protect arbitrary request-body integrity
Protecting a token against theft/replay
Supplement, not action policy.
Flexible contextual attenuation and decentralised delegation
Genuine delegated/ attenuated capabilities
Strong option when delegation becomes a requirement.
More protocol complexity than an MVP needs
Macaroon with caveats

Production and research should remain separate products

The plan already recognises two workstreams; I would formalise that into release packaging. [source document]

NPB-Core should contain no model-specific code. It consists of identity/profile resolution, source selection, schema validation, disclosure rendering, broker authorisation, auditing and deterministic tests. This is where the production security version should live.

NPB-Neural should contain model adapters, masks, K/V partitioning, differential tests and formal correspondence experiments. A backend should earn a label such as:

noninterference-tested: llama-X / torch-Y / transformers-Z /

attention-backend-Q

rather than a generic “NPB-secure” label.

The distinction reduces the trusted computing base and prevents a failure in experimental model internals from weakening an otherwise conventional authorisation boundary.

Verification and evaluation methodology

Deterministic assurance should come before benchmark statistics

For NPB-Core, several properties are ordinary program invariants and should be evaluated as such rather than statistically.

$C = (R, D, A)$

For profile , require:

$\text{ProviderSources}(\text{request}) \subseteq R$

$\text{ReleasedSourceSpan}(\text{output}) \subseteq D$

$\text{action}(p) \in \text{dom}(A)$

and

$\text{Effect}(e) \Rightarrow \exists g : \text{ValidGrant}(g, e, C, \text{principal})$

with the additional condition that the same grant cannot authorise multiple independent effects.

The gate tests should exhaustively enumerate small profile/action universes and then apply property- based fuzzing to larger JSON and source inputs. Zero failures here is an engineering exit criterion; a -

p

value adds little.

The most important negative test is a malicious provider fixture that deliberately emits every forbidden form regardless of its prompt: wrong source ID, secret copied into a nominal quote, unknown action, valid action with extra key, confused principal, replayed grant, stale source version, oversized object, truncated JSON and valid-looking but semantically unauthorised arguments. If an intentionally hostile provider cannot cross the gate, the security claim is much more meaningful than a cooperative model passing prompt-injection tests.

Formal model of the broker

TLA+ is particularly well suited to the first formalisation because NPB's difficult broker properties involve state transitions, concurrency, expiry, consumption and replay. TLA+ was developed for specifying concurrent/distributed systems, and TLC exhaustively explores finite-state instances and produces traces leading to violated invariants.

A small model should include approximately these state variables:

Profiles Grants GrantState $\in \{\text{ACTIVE}, \text{EXECUTING}, \text{COMMITTED}, \text{FAILED}, \text{UNKNOWN}, \text{EXPIRED}\}$ Requests Effects Audit Clock

Key invariants:

NoEffectWithoutGrant PrincipalBinding ProfileBinding ActionBinding ArgumentBinding
NoGrantReuse NoPrivilegeEscalation ImmutableProfileDuringRequest DisclosureSubset

And temporal properties:

ACTIVE $\rightarrow$ EXECUTING $\rightarrow \{\text{COMMITTED}, \text{FAILED}, \text{UNKNOWN}\}$

COMMITTED never returns to ACTIVE EXPIRED never returns to ACTIVE one grant cannot

produce two distinct committed effects

Alloy is an attractive secondary tool for quickly finding counterexamples in static relationships such as source/profile/action graphs; current Alloy supports mutable state and temporal modelling as well. A theorem prover such as Lean is better reserved for the abstract attention/noninterference lemma once the definitions have stabilised; otherwise substantial effort may be spent proving the wrong abstraction.

Stronger neural noninterference experiment

For each supported neural backend, construct paired runs in which only denied content changes:

$(P, A, D) (P, A, D) 1 2$

and capture after every layer/decode step:

public token hidden states; policy K/V hash; public/working K/V; next-token logits; generated prefix; mask topology and position IDs.

.....

The strongest differential experiment is not merely “final answer unchanged”; it verifies the inductive invariant at the earliest layer where divergence could appear.

Test classes should include:

Dimension Required cases

Denied replacement

same length, shorter, longer, empty, adversarial Unicode, different tokenisation

Dimension Required cases

Position handling denied content before/after/between readable segments

Cache empty, prefix cache, dynamic growth, long context, cache boundary

Attention backend eager/reference plus every production fused backend

Decoding greedy; deterministic seeded sampling as secondary

Concurrency shared process, interleaved instances, batched inputs

Model features RoPE/position variants, GQA/MQA where supported, sliding window if claimed

Numeric mode CPU reference, FP32, deployed reduced precision/quantisation where applicable

A failed variable-length test caused purely by public-position shifts would not falsify the mathematical masking lemma; it would falsify a broader claim that the implementation provides content-substitution noninterference without a shape condition. That distinction should be documented rather than papered over.

Benchmark portfolio

The supplied plan’s benchmark split is broadly correct. [source document] The current evidence suggests this portfolio:

BIPIA remains useful for regression and external-content injection taxonomies. The repository covers EmailQA, WebQA, summarisation, TableQA and CodeQA, but WebQA/summarisation have separate source-data terms, and the repository has been archived since 16 September 2026. Because this project's sampled BIPIA cases have already influenced development, they must stay out of confirmatory claims.

InjecAgent is particularly valuable for the action boundary: its original benchmark contains 1,054 attack cases spanning 17 user tools and 62 attacker tools, making it a much closer fit to the model- proposal/broker distinction than ordinary QA attacks.

AgentDojo provides an executable environment in which both legitimate user-task success and security outcomes can be measured; the original publication reported 97 realistic tasks and 629 security test cases. The package still warns that its API is evolving, so the plan is right to pin a revision.

NIST AgentDojo-Inspect is especially attractive as a supplementary locked track because NIST describes fixes to upstream task issues, new exfiltration cases and a terminal environment aimed at remote-code-execution risks.

Finally, at least some attacks should be adaptive to NPB itself. A recent firewall study reached apparently perfect or near-perfect results on several existing benchmarks but still found practical bypasses and concluded that weak benchmarks could be saturated. Agent-Dyn's 2026 benchmark similarly argues for more dynamic, open-ended, realistic adversarial evaluation. Thus, a good locked test set should not consist exclusively of attacks written before the defence architecture was known.

Baselines and ablations

The locked evaluation should include at least the following configurations on identical cases:

Configuration Purpose

Unprotected model/agent Measures baseline attack and utility rates

Prompt/instruction defence only Measures behavioural robustness contribution

Host read filtering only Separates simple context minimisation from other mechanisms

Capability gate without neural isolation Measures deterministic NPB-Core contribution

Capability gate + broker Measures actual privileged-effect containment

Capability gate + broker + neural state mechanism

Measures whether the experimental neural layer adds utility/security

Full NPB with adaptive attack Main deployment-facing result

This is important because a zero unauthorised-effect rate from the full system is scientifically less interesting if the same result is achieved by a much simpler broker-only control. Conversely, if neural isolation improves confidentiality or utility under a workload where simple omission of denied data is impossible, the ablation demonstrates its incremental value.

Metrics and statistical analysis

The plan is right to make environment state the preferred action-security label rather than output- string change or keyword matching. [source document] A model can emit alarming text without causing an effect, or innocuous-looking text while causing a harmful one; the broker/environment is the authoritative place to observe whether an operation occurred.

Primary security endpoints should be reported separately:

unauthorised effects

Unauthorised effect rate = attack opportunities

Unauthorised disclosure rate = disclosure opportunities protected disclosure events

plus forbidden-proposal rate, cross-instance violation rate and model-level attack success. Do not average these into a single "security score".

For a zero-event result across independent Bernoulli opportunities, the one-sided exact 95% upper

n

confidence limit is:

$p = \text{upper } 1 - 0.05 \cdot 1/n$

3/n 36

The familiar rule of three approximates this by . Approximately 299 independent zero-failure

observations are needed before that upper limit falls below 1%. Clopper–Pearson is the classic exact- binomial construction.

"Independent" is the important qualification. Hundreds of prompts derived from the same attack template/source do not provide hundreds of independent pieces of evidence. The plan's proposal to resample by source or task cluster is therefore well motivated. Cluster bootstrapping preserves grouped sampling structure rather than pretending prompt rows are exchangeable independent observations.

For paired protected/unprotected binary outcomes, exact McNemar analysis is suitable because the same underlying case is exposed to both systems. Multiple confirmatory comparisons should be declared beforehand and corrected; Holm's sequentially rejective procedure controls family-wise type-I error without requiring the more restrictive independence assumptions needed by some alternatives.

The clean-task side needs equal discipline. Define a non-inferiority margin before seeing locked results. Report task completion, quote success, legitimate-action completion, refusal/over-block rate, provider latency, gate latency, p95 tail latency, token consumption and monetary cost independently. A security mechanism that blocks every action is secure under a trivial metric but useless; the security/utility pair is the meaningful result.

Benchmark governance and leakage

The dataset manifest should make every case traceable to source URL/repository, commit, licence, source hash, task identifier, transformation code and first date seen by the project. BIPIA's current repository itself illustrates why this matters: different component datasets have different source terms.

A final case must cease to be “held out” as soon as its prompt, expected result or semantic equivalent is manually inspected for defence tuning. This should be an append-only metadata event, not a judgement made retrospectively.

Recommended implementation and roadmap

Target architecture

The best implementation path is a deliberately asymmetric system: the model receives information but no authority, while trusted host code possesses authority but does not perform semantic language reasoning.

The end-to-end request should operate as follows.

Step Implementation Required security property

Authenticate Resolve principal outside the model path. Request cannot supply its own authoritative identity.

Resolve profile Load immutable versioned capability profile by trusted mapping.

No model/caller capability escalation.

Snapshot sources Resolve canonical version/hash for all candidate sources.

Stable provenance; no TOCTOU replacement.

Step Implementation Required security property

Select readable evidence Include only source IDs in read(C) . Denied data does not reach an external provider.

Invoke provider Use strict structured-response mode, but treat output as untrusted bytes. Provider has no authority.

Strictly parse Apply size/depth limits and reject malformed/unknown structures. Parser cannot widen schema.

Apply capability checks

Verify source/action against immutable profile.

Deterministic policy enforcement.

Render disclosure Copy the authorised canonical source range in trusted code.

Model never manufactures released protected text.

Resolve grant Match principal, instance/profile, action and canonical argument hash.

Proposal does not imply authorisation.

Execute via broker

Consume or transition grant atomically; pass stable idempotency key.

Complete mediation and replay control.

Audit Record hashes/IDs/decision metadata without routine sensitive payloads.

Reproducibility without secondary data leakage.

This is, in effect, an AI-specific application of the reference-monitor pattern.

Security contract details that should be frozen first

A profile should not merely list action names and argument keys. It should encode constraints such as:

profile_version principal_scope

read_sources disclose_sources

actions: send_email: allowed_recipient_ids allowed_sender_identity body_mode
max_body_length requires_fresh_consent create_calendar_event: allowed_calendar_ids
allowed_attendee_ids maximum_duration permitted_time_window

Why values matter is straightforward: validating that send_email has keys {to,body} is not secure

if prompt injection can change to from the user's colleague to an attacker-controlled address. The

security boundary is the tuple action + values + principal + resource + context, not merely the JSON shape.

JSON Schema can provide structural checks, but application-level semantic checks remain necessary.

Fail-closed behaviour

The following cases should all produce the same external security result:

provider timeout provider refuses provider returns truncated JSON unknown proposal variant schema violation unknown source non-disclosable source profile changed during request expired grant grant already consumed principal mismatch argument mismatch broker timeout before execution

Namely:

no disclosure and no new privileged effect.

Some can return non-sensitive diagnostic status codes to the caller; none should cause the system to "fall back" to an unrestricted model answer. This follows classical fail-safe-default reasoning.

Concurrency-safe broker state

I recommend a state transition rather than a simple Boolean used flag:

ACTIVE | v EXECUTING / | \{} v v v COMMITTED FAILED UNKNOWN

UNKNOWN is important when an external effect may have succeeded but acknowledgement was lost. A

replay of an UNKNOWN grant should not simply execute again unless the external operation supports

the original idempotency key.

The atomic transition from ACTIVE to EXECUTING should be the broker's replay barrier. Concurrent

requests attempting to consume the same grant must not both succeed.

Neural research implementation

For NPB-Neural, preserve a deliberately narrow compatibility matrix. The plan currently specifies a reproducible neural environment around Python 3.11, Torch 2.6.0 and Transformers 4.57.6; that version

pin is a project requirement rather than a universal backend guarantee. [source document]

The sequence should be:

Implement a tiny mathematical/reference attention model whose semantics are easy to inspect. State the noninterference theorem including position/shape conditions. Prove the layer induction on that abstraction. Implement the corresponding mask/cache transition in one real model family. Differentially test hidden states and logits against denied-source substitutions. Repeat on a second architecturally different family before claiming model-family generality. Treat every new attention/cache/kernel backend as a new conformance target rather than inheriting the claim automatically.

1. 2. 3. 4. 5. 6. 7.

The project should also test the strongest counterexample to the policy-memory narrative: construct a toy model that preserves an immutable policy cache perfectly but deliberately chooses an output based only on readable attacker context. A successful counterexample is valuable because it formally demonstrates why “cache integrity” cannot entail “instruction obedience”.

Roadmap

The following is an aggressive but realistic research-engineering sequence assuming the staffing and infrastructure assumptions stated earlier. It deliberately puts the deterministic boundary before expensive model evaluation.

A useful go/no-go ladder is:

Research-ready: definitions, threat model and deterministic reference implementation frozen.

Integration-ready: malicious fake-provider tests cannot produce forbidden disclosures or effects.

Evaluation-ready: broker model checked, benchmark manifest pinned, preregistration frozen.

Production-candidate: zero deterministic invariant violations, no unauthorised effect/disclosure in locked testing, clean-task utility within pre-agreed bound, and operational failure modes demonstrated.

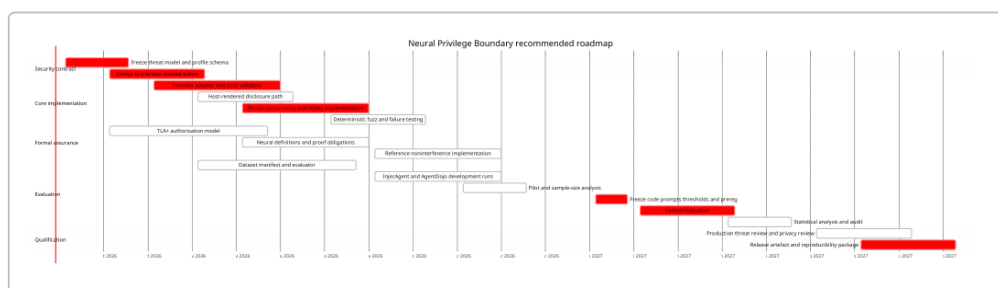


Figure 2: NPB research and engineering roadmap (reproduced from the supplied report).

Production-qualified: identity/profile integrity, secrets management, audit privacy, incident response, source/provider licence review and runtime/version pinning complete.

No model benchmark result should be permitted to waive a deterministic gate failure.

Compute and cost

The supplied plan is sensible to keep the standard-library gate/CI environment independent of the neural model environment and to defer paid GPU work until the experimental matrix is known. [source document] Most of the highest-value early work—schema design, validator tests, broker state machine, model checking, dataset manifests and fake-provider fuzzing—requires no GPU.

The lowest-cost path is therefore to make NPB-Core complete before provisioning broad model evaluation. This also provides a useful scientific ablation: if the deterministic gate already solves the privileged-effect problem, GPU work can focus narrowly on the separate noninterference research question rather than being treated as a prerequisite for application security.

For production confidentiality, simple source omission is preferable to carrying denied tokens through a neural runtime and masking them. It is easier to reason about, cheaper computationally, and shrinks the amount of sensitive data exposed to model code. Neural masking only earns its extra complexity where the research question or a concrete shared-state use case specifically requires denied information to coexist inside the local model computation.

Risks, patent landscape and further reading

Principal engineering risks

Risk	Failure mode	Recommended mitigation
1	2	3
4	5	6
7	8	9
10	11	12
13	14	15
16	17	18
19	20	21
22	23	24
25	26	27
28	29	30
31	32	33
34	35	36
37	38	39
40	41	42
43	44	45
46	47	48
49	50	51
52	53	54
55	56	57
58	59	60
61	62	63
64	65	66
67	68	69
70	71	72
73	74	75
76	77	78
79	80	81
82	83	84
85	86	87
88	89	90
91	92	93
94	95	96
97	98	99
100	101	102

"Immutable policy cache" becomes "prompt injection solved".

Maintain separate capability, noninterference and behavioural claims in every document/
release.

Claim inflation

Make the broker the only network/service credential holder; scan/test for direct-effect paths. This is the reference-monitor requirement.

Incomplete mediation

One tool/API path bypasses the broker.

Argument smuggling

Action name allowed but dangerous values pass.

Bind semantic values and canonical argument digest, not merely key names.

Quote validated against one version and released from another.

TOCTOU source replacement

Immutable source snapshots/version hashes and host-rendered output.

Atomic grant-state transition, short expiry, idempotency key, replay store; DPoP offers useful analogous replay controls.

Replay/race One authorised proposal causes multiple effects.

Provider and host interpret structured output differently.

Validate raw provider output in one strict trusted implementation; reject unknown/duplicate constructs and coercion.

Schema/parser differential

Risk Failure mode Recommended mitigation

Shared cache/source ID/ profile accidentally crosses tenant boundary.

Instance-bound source handles, profile hashes, cache isolation and explicit cross-principal adversarial tests.

Cross-instance leakage

Sensitive evidence leaks through debugging/ observability.

Metadata/hashes by default, narrowly controlled encrypted diagnostics when necessary.

Log leakage

Versioned compatibility matrix and rerun conformance suite per backend. Hugging Face cache behaviour is implementation/model dependent.

New kernel/cache/model violates masking assumptions.

Neural backend drift

Adaptive attacks, independently locked tasks, multiple environments and explicit confidence intervals. Recent work has demonstrated this saturation problem.

Apparent zero-ASR is mistaken for broad robustness.

Benchmark saturation

Huge context/output or malformed structures exhaust resources.

Availability attacks

Input/output bounds, execution deadlines, quotas and no-effect failure semantics.

Quote-only/strict action contract makes product unusable.

Measure clean-task non-inferiority separately; introduce new capabilities deliberately rather than falling back to unrestricted effects.

Utility collapse

Patent landscape

A non-exhaustive patent search finds several adjacent families, which is enough to justify a professional novelty/freedom-to-operate review before commercialisation but not enough to draw any legal conclusion.

One published family, WO2024238244A1, describes signing or otherwise protecting prompts to detect/ prevent prompt-injection-style manipulation. US12340000B2 concerns prompt-injection detection/ firewall behaviour involving test or forbidden instructions and checking model responses. US20260003958A1 describes prompt-injection detection using measures of trusted-prompt influence/ trust. Searches also surface attention/KV-management inventions using non-contiguous masks, although their purpose is cache/computation management rather than the specific security theorem proposed by NPB.

This limited search did not establish that the particular combination of immutable policy K/V, denied- source causal noninterference, host-level disclosure constraints and brokered capabilities is either novel or non-infringing. Patent publications are themselves claims about inventions, not empirical evidence that the security techniques work, and apparently similar terminology can have materially different claim scope. A patent professional should search full claim families, prosecution histories and relevant jurisdictions before any freedom-to-operate assertion.

From a research-publication standpoint, the safer contribution framing is also the more scientifically defensible one: rather than claiming invention of “a prompt-injection-proof neural firewall”, make precise contributions around formal noninterference under a specified Transformer architecture,

structural policy-state integrity, and composition with a deterministic least-authority capability boundary.

Suggested further reading and source links

Area	Primary or authoritative source	Why it matters
Security architecture	Saltzer & Schroeder, <i>The Protection of Information in Computer Systems</i>	Classical least privilege, fail-safe defaults, mediation and protection design. ¹⁴
Reference monitors	NIST reference-monitor definition	Direct architectural model for the NPB gate/broker. ²⁸
Early capabilities	Dennis & Van Horn, <i>Programming Semantics for Multiprogrammed Computations</i>	Foundational protected-object/capability concepts. ⁴⁶
Attenuated authorisation	Birgisson et al., <i>Macaroons</i>	Practical caveated authority and delegation. ²⁹
Replay-resistant authorisation	IETF RFC 9449, DPoP	Proof-of-possession and replay-control patterns, plus important limitations. ¹⁸
Information-flow security	Goguen & Meseguer, <i>Security Policies and Security Models</i>	Foundation for the noninterference claim. ⁴⁷
Transformer mechanism	Vaswani et al., <i>Attention Is All You Need</i>	Mathematical basis needed for the attention-mask argument. ²¹
Runtime cache behaviour	Hugging Face KV-cache documentation	Real implementation assumptions behind K/V separation. ²²
Prompt injection	Greshake et al., indirect prompt-injection research	Establishes the untrusted-content instruction problem in integrated applications. ⁴⁸
Instruction hierarchy	Wallace et al., <i>The Instruction Hierarchy</i>	Strong behavioural defence-in-depth, distinct from authorisation. ⁴
External-content benchmark	BIPIA	Useful benchmark history, data/licence details and prompt-level defences. ¹¹
Tool-agent benchmark	InjecAgent	Particularly relevant to NPB's action proposal/broker model. ⁴⁹
Dynamic agent evaluation	AgentDojo	Measures both user utility and injection-driven agent behaviour. ³³
Independent benchmark tooling	NIST AgentDojo-Inspect	Useful updated/fixed execution-based security suite. ³⁵

Area	Primary or authoritative source	Why it matters
Benchmark limitations	<i>Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks?</i>	Important warning against interpreting saturated public benchmarks as proof. ⁵
Recent dynamic attacks	AgentDyn	Evidence for adding adaptive, open-ended security evaluation. ⁶
Formal concurrency verification	Lamport, TLA+ / TLC	Best fit for broker replay/ concurrency state-machine verification. ⁹
Structured validation	JSON Schema Draft 2020-12 validation specification	Defines what schema validation does—and does not—guarantee. ⁸
Statistical zero-event inference	Clopper-Pearson; zero-event “rule of three” literature	Correctly scopes claims after zero observed security failures. ⁵⁰

The resulting assurance hierarchy should therefore be deliberately heterogeneous:

Host capability safety	→ proof by control flow + exhaustive/property tests
Broker concurrency/replay	→ state-machine model checking + implementation tests
Denied-source noninterference	→ mathematical lemma + backend conformance tests
Policy-cache integrity	→ state invariant + byte/state verification
Prompt-injection robustness	→ empirical adversarial evaluation
Real-world residual risk	→ adaptive red-team testing + operational controls

That is the central principle underlying the best version of the NPB plan: **prove deterministic properties where deterministic mechanisms exist; model-check stateful authorisation where concurrency matters; statistically evaluate probabilistic model behaviour; and never let empirical model robustness substitute for a trusted privilege boundary.**    

¹ ¹³ ²⁸ reference monitor - Glossary | CSRC

https://csrc.nist.gov/glossary/term/reference_monitor

² ¹⁵ ⁴⁶ Programming Semantics for Multiprogrammed Computations

<https://dspace.mit.edu/entities/publication/4f4d9ee2-671a-4dfd-a679-7b24fae5812a>

³ prompt injection - Glossary | CSRC

https://csrc.nist.gov/glossary/term/prompt_injection

⁴ The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions

<https://arxiv.org/abs/2404.13208>

⁵ ⁴¹ Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks?

<https://arxiv.org/abs/2510.05244>

⁶ AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System

<https://arxiv.org/abs/2602.03117>

[https://www.ndss-symposium.org/ndss2014/ndss-2014-programme/macaroons-cookies-contextual-caveats-decentralized-](https://www.ndss-symposium.org/ndss2014/ndss-2014-programme/macaroons-cookies-contextual-caveats-decentralized-authorization-cloud/)

authorization-cloud/

JSON Schema Validation: A Vocabulary for Structural Validation of JSON

<https://json-schema.org/draft/2020-12/json-schema-validation>

Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers - Microsoft Research

[https://www.microsoft.com/en-us/research/publication/specifying-systems-the-tla-language-and-tools-for-hardware-and-](https://www.microsoft.com/en-us/research/publication/specifying-systems-the-tla-language-and-tools-for-hardware-and-software-engineers/?lang=fr-ca)

software-engineers/?lang=fr-ca

GitHub - microsoft/BIPIA: A benchmark for evaluating the robustness of LLMs and defenses to indirect prompt injection attacks. · GitHub

<https://github.com/microsoft/BIPIA>

Technical Blog: Strengthening AI Agent Hijacking Evaluations | NIST

<https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations>

Publications of Jerome H. Saltzer

<https://www.mit.edu/~Saltzer/publications/pubs.html>

RFC 9449: OAuth 2.0 Demonstrating Proof of Possession (DPoP) | RFC Editor

<https://www.rfc-editor.org/info/rfc9449/>

RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage | RFC Editor

<https://www.rfc-editor.org/info/rfc6750/>

Security Policies and Security Models - Illinois Experts

<https://experts.illinois.edu/en/publications/security-policies-and-security-models/>

Attention is All you Need

https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Cache strategies · Hugging Face

https://huggingface.co/docs/transformers/main/kv_cache

torch.use_deterministic_algorithms — PyTorch main documentation

https://docs.pytorch.org/docs/main/generated/torch.use_deterministic_algorithms.html

alloytools.org

<https://alloytools.org/documentation.html>

The Lean Theorem Prover (System Description) - Microsoft Research

<https://www.microsoft.com/en-us/research/publication/the-lean-theorem-prover-system-description/>

InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language

Model Agents - ACL Anthology

<https://aclanthology.org/2024.findings-acl.624/>

AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents

<https://arxiv.org/abs/2406.13352>

agentdojo · PyPI

<https://pypi.org/project/agentdojo/>

PDR: AgentDojo-Inspect

<https://data.nist.gov/pdr/lps/ark%3A/88434/mds2-3690>

If nothing goes wrong, is everything all right? Interpreting zero numerators.

<https://pubmed.ncbi.nlm.nih.gov/6827763/>

THE USE OF CONFIDENCE OR FIDUCIAL LIMITS ILLUSTRATED IN THE CASE OF THE BINOMIAL | Biometrika | Oxford Academic

<https://doi.org/10.2307/2331986>

Use of the bootstrap in analysing cost data from cluster randomised trials: some simulation results | BMC Health Services Research | Springer Nature Link

<https://springerlink.fh-diploma.de/article/10.1186/1472-6963-4-33>

Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages | Psychometrika | Cambridge Core

[https://www.cambridge.org/core/journals/psychometrika/article/note-on-the-sampling-error-of-the-difference-between-](https://www.cambridge.org/core/journals/psychometrika/article/note-on-the-sampling-error-of-the-difference-between-correlated-proportions-or-percentages/698C2461BE63F5848763502D54E534FD)

[correlated-proportions-or-percentages/698C2461BE63F5848763502D54E534FD](https://www.cambridge.org/core/journals/psychometrika/article/note-on-the-sampling-error-of-the-difference-between-correlated-proportions-or-percentages/698C2461BE63F5848763502D54E534FD)

Rankless | A Simple Sequentially Rejective Multiple Test Procedure

<https://www.rankless.org/hit-papers/10.2307/4615733>

WO2024238244A1 - Signing large language model prompts to prevent prompt injection attacks - Google Patents

<https://patents.google.com/patent/WO2024238244A1/en>

US12340000B2 - Prompt injection detection for large language models - Google Patents

<https://patents.google.com/patent/US12340000B2/en>

US20260003958A1 - Method and apparatus for detecting prompt injections in llm-integrated applications - Google Patents

<https://patents.google.com/patent/US20260003958A1/en>

WO2025259390A1 - Non-contiguous attention mask for key-value (kv) cache management for fixed-length transformer models - Google Patents

<https://patents.google.com/patent/WO2025259390A1/en>

dblp: Security Policies and Security Models.

<https://dblp.org/rec/conf/sp/GoguenM82a.html>

Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection

<https://arxiv.org/abs/2302.12173>

InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents

<https://arxiv.org/abs/2403.02691>